DuckDB for analytics

Lecture 18

Dr. Colin Rundel

Data and plans

Data

One week of trains trips in the Netherlands, from the DuckDB blog's railway data. The file is also available from https://blobs.duckdb.org/train_services.parquet.

```
D FROM 'train_services.parquet';
```

DuckDB

service_id int64	date date	…	departure_time timestamp	arrival_time timestamp
11196117	2023-05-15	…	2023-05-15 00:00:00	NULL
11196117	2023-05-15	…	2023-05-15 00:13:00	2023-05-15 00:13:00
11196117	2023-05-15	…	2023-05-15 00:29:00	2023-05-15 00:21:00
.	.	…	.	.
.	.	…	.	.
.	.	…	.	.
11239115	2023-05-21	…	2023-05-21 11:03:00	NULL
11239115	2023-05-21	…	2023-05-21 11:16:00	2023-05-21 11:14:00
11239115	2023-05-21	…	NULL	2023-05-21 11:32:00

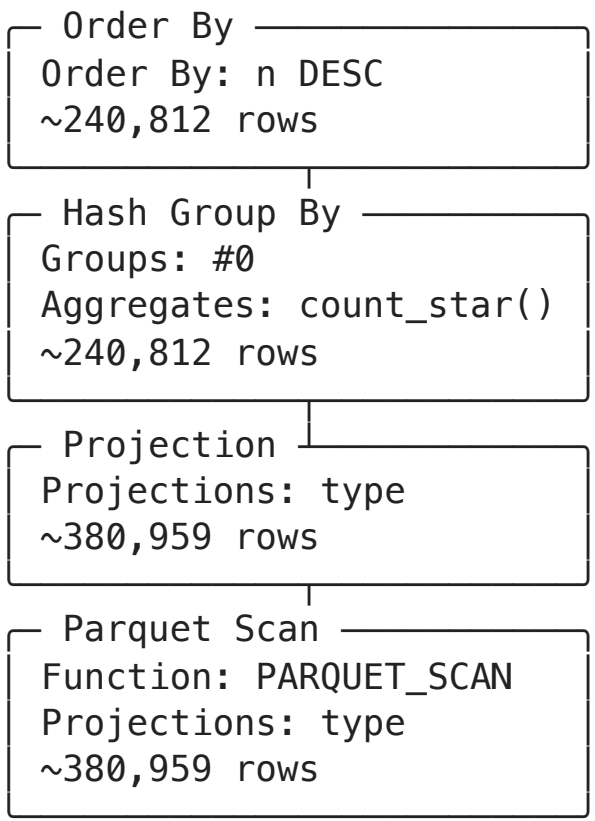
380959 rows (6 shown) 8 columns (4 shown)

EXPLAIN

Prefixing a query with `EXPLAIN` returns the plan DuckDB will run rather than the result. The plan is a tree of operators, read from the bottom (the scans) to the top.

DuckDB

```
D EXPLAIN
  SELECT type, count(*) AS n
  FROM 'train_services.parquet'
  GROUP BY ALL
  ORDER BY n DESC;
```



Reading a plan

Operator	Description
`Seq Scan`, `Parquet Scan`	Read a table or file
`Filter`	Apply a `WHERE` condition
`Projection`	Compute and select output columns (`SELECT`)
`Hash Join`	Join by building a hash table on one input and probing with the other
`Hash Group By`	Group rows and compute aggregates with a hash table (`GROUP BY`)
`Ungrouped Aggregate`	A single aggregate over all rows
`Order By`	Sort rows
`Top N`	`ORDER BY` with a `LIMIT`, keeps only the needed rows

Each box also lists the columns it needs (`Projections`), any filters it applies, and the estimated number of rows, which is what the optimizer uses to choose between plans.

EXPLAIN ANALYZE

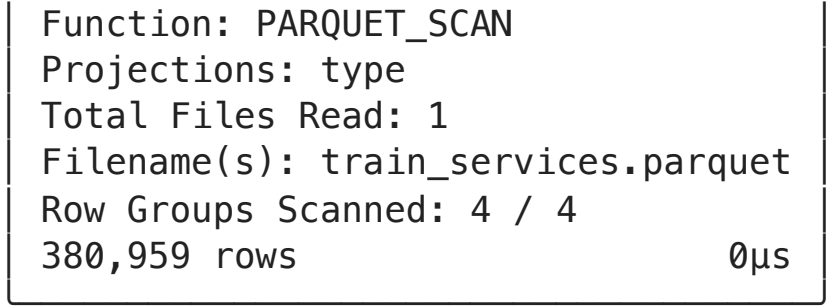
The same query with `EXPLAIN ANALYZE`, which runs it and adds the actual rows and time per operator and a summary of the data read.

DuckDB

```
D EXPLAIN ANALYZE
  SELECT type, count(*) AS n
  FROM 'train_services.parquet'
  GROUP BY ALL
  ORDER BY n DESC;
```

```
┌─ Summary ─────────┐
│ Total Time: 0.0052s │
│ Data Read:  45.0 kB │
└─────────────────────┘

┌─ Order By ──────────────────────────┐
│ Order By: n DESC                    │
│ 15 rows                       0µs   │
└─────────────────────────────────────┘
                  │
┌─ Hash Group By ─┴───────────────────┐
│ Groups: #0                          │
│ Aggregates: count_star()            │
│ 15 rows                     10.0ms  │
└──────────────┬──────────────────────┘
┌─ Projection ─┴──────────────────────┐
│ Projections: type                   │
│ 380,959 rows                  0µs   │
└──────────────┬──────────────────────┘
┌─ Table Scan ─┴──────────────────────┐
```


Function: PARQUET_SCAN
Projections: type
Total Files Read: 1
Filename(s): train_services.parquet
Row Groups Scanned: 4 / 4
380,959 rows
0µs

Parquet

Row vs column storage

Row oriented (CSV, most databases)

```
name,  salary, dept
Alice, 52000,  Accounting
Bob,   40000,  Accounting
Carol, 30000,  Sales
```

Reading one column means reading every row, and every value must be parsed from text.

Column oriented (Parquet, DuckDB)

```
name:   [Alice, Bob, Carol, ...]
salary: [52000, 40000, 30000, ...]
dept:   [Accounting, Accounting, Sales]
```

Reading one column skips the others entirely, and values of one type compress well together.

Analytical queries touch a few columns of many rows, which is exactly the access pattern column storage is good at. Parquet is the standard columnar file format, used by Spark, Arrow, `pandas`, `polars`, DuckDB, and most cloud warehouses.

Inside a Parquet file

A Parquet file is split into row groups (DuckDB writes 122,880 rows per group by default). Within a row group each column is stored separately as a column chunk, which is compressed and encoded on its own.

The footer holds the schema plus, for every column chunk, its location, size, and statistics (minimum, maximum, and null count).

Structure	What a reader can do with it
column chunks	read only the columns the query needs (projection pushdown)
row group stats	skip row groups whose min / max cannot match a filter
typed, encoded	no parsing, dictionary and run length encodings shrink data
footer	learn the schema and plan without reading the data

File metadata

DuckDB exposes the footer through table functions. This file was written in date order, so each row group covers a narrow range of dates.

DuckDB

```
D SELECT num_rows, num_row_groups, format_version, file_size_bytes
  FROM parquet_file_metadata('train_services.parquet');
```

num_rows int64	num_row_groups int64	format_version int64	file_size_bytes uint64
380959	4	1	1598231

DuckDB

```
D SELECT row_group_id, row_group_num_rows, stats_min, stats_max, compression
  FROM parquet_metadata('train_services.parquet')
  WHERE path_in_schema = 'date';
```

row_group_id int64	row_group_num_rows int64	stats_min varchar	stats_max varchar	compression varchar
0	122880	2023-05-15	2023-05-17	ZSTD
1	122880	2023-05-17	2023-05-19	ZSTD
2	122880	2023-05-19	2023-05-21	ZSTD
3	12319	2023-05-21	2023-05-21	ZSTD

Projection pushdown

The scan reads only the columns a query refers to, and the summary box reports how much of the file that was.

DuckDB

```
D EXPLAIN ANALYZE
  SELECT sum(trip_distance)
  FROM 'yellow_tripdata_2024-01.parquet';
```

```
┌─ Summary ─────────────┐
│ Total Time: 0.0125s   │
│ Data Read:   4.1 MB   │
└───────────────────────┘

┌─ Ungrouped Aggregate ─┐
│ Aggregates: sum(#0)   │
│ 1 row            0µs  │
└───────────┬───────────┘
┌─ Projection ──┴───────┐
│ Projections: trip_distance
│ 2,964,624 rows   0µs  │
└───────────┬───────────┘
┌─ Table Scan ──┴───────┐
│ Function: PARQUET_SCAN
│ Projections: trip_distance
│ Total Files Read: 1   │
│ Filename(s):          │
│ yellow_tripdata_2024-01.parquet
│ Row Groups Scanned: 3 / 3
```

```
┌──────────────┐
│ 2,964,624 rows │
│ 20.0ms │
└──────────────┘
```

`yellow_tripdata_2024-01.parquet` is one month of NYC taxi trips, 3 million rows and 19 columns in 50 MB.

More columns, more bytes

DuckDB

```
D EXPLAIN ANALYZE
  SELECT sum(trip_distance), sum(total_amount), sum(tip_amount)
  FROM 'yellow_tripdata_2024-01.parquet';
```

```
┌─ Summary ─────────────┐
│ Total Time: 0.0142s   │
│ Data Read:  12.4 MB   │
└───────────────────────┘

┌─ Ungrouped Aggregate ─┐
│ Aggregates: sum(#0), sum(#1),
│             sum(#2)
│ 1 row                 10.0ms
└─────────────┬─────────┘┌─────────────┐
┌─ Projection ┴──────────┐
│ Projections: trip_distance,
│              total_amount,
│              tip_amount
│ 2,964,624 rows              0µs
└─────────────┬─────────┘┌─────────────┐
┌─ Table Scan ┴──────────┐
│ Function: PARQUET_SCAN
│ Projections: trip_distance,
│              total_amount,
│              tip_amount
│ Total Files Read: 1
│ Filename(s):
│ yellow_tripdata_2024-01.parquet
│ Row Groups Scanned: 3 / 3
```

```
┌──────────────┐
│ 2,964,624 rows │
│    20.0ms    │
└──────────────┘
```

A bare `count(*)` reads no columns at all, the row count comes from the footer and the plan is empty.

Filter pushdown and zone maps

A filter is pushed into the scan. Before reading a row group DuckDB compares the filter against the row group's min and max, and skips it when no row can match.

DuckDB

```
D EXPLAIN ANALYZE
  SELECT count(*)
  FROM 'train_services.parquet'
  WHERE date = '2023-05-21';
```

```
┌─ Summary ─────────────┐
│ Total Time: 0.0021s   │
│ Data Read:  16.5 kB   │
└───────────────────────┘

┌─ Ungrouped Aggregate ─────────────────────┐
│ Aggregates: count_star()                  │
│ 1 row                              0µs    │
└─┬─────────────────────────────────────────┘
  │
┌─┴─ Table Scan ────────────────────────────┐
│ Function: PARQUET_SCAN                    │
│ Filters: date = '2023-05-21'::DATE        │
│ Total Files Read: 1                       │
│ Filename(s): train_services.parquet       │
│ Row Groups Scanned: 2 / 4                 │
│ 44,679 rows                        0µs    │
└───────────────────────────────────────────┘
```

DuckDB

```
D EXPLAIN ANALYZE
  SELECT count(*)
  FROM 'train_services.parquet'
  WHERE station_name = 'Utrecht Centraal';
```

```
┌─ Summary ─────────────┐
│ Total Time: 0.0014s   │
│ Data Read: 141.3 kB   │
└───────────────────────┘

┌─ Ungrouped Aggregate ─────────────────────┐
│ Aggregates: count_star()                  │
│ 1 row                              0µs    │
└─┬─────────────────────────────────────────┘
  │
┌─┴─ Table Scan ────────────────────────────┐
│ Function: PARQUET_SCAN                    │
│ Filters:                                  │
│ station_name = 'Utrecht Centraal'         │
│ Total Files Read: 1                       │
│ Filename(s): train_services.parquet       │
│ Row Groups Scanned: 4 / 4                 │
│ 7,663 rows                         0µs    │
└───────────────────────────────────────────┘
```

Sort order matters

Every row group spans `'s-Hertogenbosch` to `Zürich HB`, so a station filter skips nothing. Rewriting the file sorted by station changes that.

DuckDB

```
D COPY (FROM 'train_services.parquet' ORDER BY station_name)
    TO 'train_services_sorted.parquet';
```

DuckDB

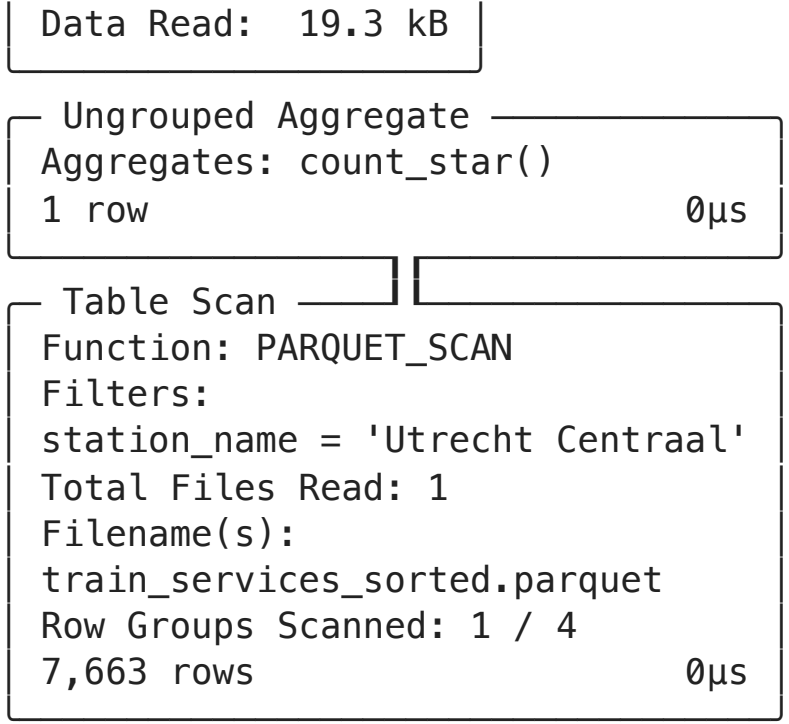
```
D SELECT row_group_id, stats_min, stats_max
  FROM parquet_metadata('train_services_sorted.parquet')
  WHERE path_in_schema = 'station_name';
```

row_group_id	stats_min	stats_max
int64	varchar	varchar
0	's-Hertogenbosch	Doetinchem
1	Doetinchem	Middelburg
2	Middelburg	Zevenaar
3	Zevenaar	Zürich HB

DuckDB

```
D EXPLAIN ANALYZE
  SELECT count(*)
  FROM 'train_services_sorted.parquet'
  WHERE station_name = 'Utrecht Centraal';
```

```
┌─ Summary ─┐
│ Total Time: 0.0008s │
```


Data Read: 19.3 kB
Ungrouped Aggregate
Aggregates: count_star()
1 row 0µs
Table Scan
Function: PARQUET_SCAN
Filters:
station_name = 'Utrecht Centraal'
Total Files Read: 1
Filename(s):
train_services_sorted.parquet
Row Groups Scanned: 1 / 4
7,663 rows 0µs

Exercise 1

Using `train_services.parquet`,

1. Write a query that counts the number of stops per train `type` on 2023-05-20 and run it with `EXPLAIN ANALYZE`. Which columns were read, how many row groups were scanned, and how much data was read?
2. Change the filter to a single `train_number` and compare. Why does this query read more of the file?

Storage formats

Three ways to store a table

`services-2024-01.csv.gz` holds every stop of every train in the Netherlands for January 2024, 1.8 million rows. We load six of its columns as a native table and write that back out as CSV and Parquet.

DuckDB

```
D .timer on
```

DuckDB

```
D CREATE TABLE services AS
    SELECT "Service:Date" AS date, "Service:Company" AS company,
           "Service:Type" AS type, "Stop:Station name" AS station,
           "Stop:Arrival delay" AS arr_delay, "Stop:Departure delay" AS dep_delay
    FROM 'services-2024-01.csv.gz';
```

```
Run Time (s): real 0.498 user 0.837734 sys 0.018456
```

DuckDB

```
D COPY services TO 'services-2024-01.csv';
```

```
Run Time (s): real 0.065 user 0.339963 sys 0.052938
```

DuckDB

```
D COPY services TO 'services-2024-01.parquet';
```

```
Run Time (s): real 0.059 user 0.292006 sys 0.023903
```

The same query, three times

DuckDB

```
D SELECT sum(arr_delay) AS delay, count(*) AS n
  FROM 'services-2024-01.csv'
  WHERE station = 'Utrecht Centraal';
```

delay int128	n int64
28703	40573

Run Time (s): real 0.040 user 0.170709 sys 0.009023

DuckDB

```
D SELECT sum(arr_delay) AS delay, count(*) AS n
  FROM 'services-2024-01.parquet'
  WHERE station = 'Utrecht Centraal';
```

delay int128	n int64
28703	40573

Run Time (s): real 0.002 user 0.013686 sys 0.002293

DuckDB

```
D SELECT sum(arr_delay) AS delay, count(*) AS n
  FROM services
  WHERE station = 'Utrecht Centraal';
```

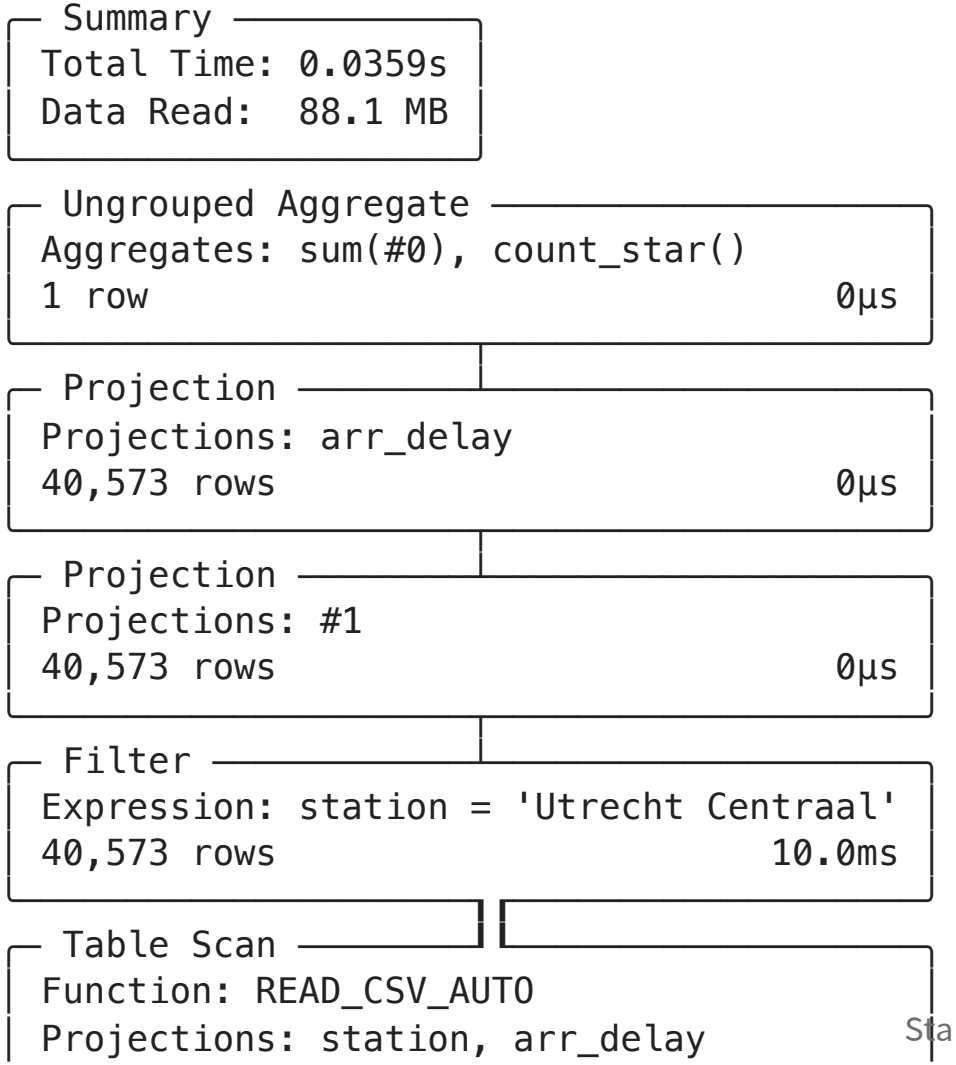
delay int128	n int64
28703	40573

Run Time (s): real 0.002 user 0.019521 sys 0.000596

Where the time goes

DuckDB

```
D EXPLAIN ANALYZE
  SELECT sum(arr_delay) AS delay, count(*) AS n
  FROM 'services-2024-01.csv'
  WHERE station = 'Utrecht Centraal';
```



```
┌──────────────────────────────────────────┐
│ Total Files Read: 1                      │
│ Filename(s): services-2024-01.csv        │
│ 1,804,898 rows                 160.0ms   │
└──────────────────────────────────────────┘
```

Reading the CSV plan

- The CSV scan parses all 1.8 million rows, nearly all of the query's time is spent there. Only two columns are kept, but every byte still has to be read and tokenized.
- The filter is a separate operator above the scan. A CSV file has no statistics, so there is nothing to push the filter into.
- The Parquet and native table scans show `Filters:` inside the scan node and only read the needed column chunks, so the same query is 20 to 30 times faster.
- The native table is faster still. DuckDB keeps additional statistics (such as distinct value counts) that make its estimates and plans better, and no file format decoding is needed.

Compression

The same table on disk in different formats,

```r
fs::file_info(c(
  "services-2024-01.csv", "services_uncompressed.parquet",
  "services-2024-01.parquet", "services_zstd.parquet", "services.duckdb"
)) |>
  select(path, size) |>
  arrange(desc(size))
```

```
# A tibble: 5 × 2
  path                                 size
  <fs::path>                    <fs::bytes>
1 services-2024-01.csv                72.53M
2 services.duckdb                      9.01M
3 services_uncompressed.parquet        6.55M
4 services-2024-01.parquet             3.39M
5 services_zstd.parquet                2.16M
```

`COPY services TO 'services_zstd.parquet' (COMPRESSION zstd)` and similar wrote the variants. Snappy is the default since every reader supports it, `zstd` is smaller at the cost of slower writes. Parquet also dictionary and run length

Partitioning

Hive partitioning writes a directory of files, one per value of the partition columns, with the value encoded in the path. A filter on a partition column then skips whole files.

DuckDB

```
D COPY services TO 'services_part' (FORMAT parquet, PARTITION_BY (company));
```

R

```
fs::dir_ls("services_part", recurse = TRUE, type = "file") |> head(4)
```

```
services_part/company=Arriva/data_0.parquet
services_part/company=Blauwnet/data_0.parquet
services_part/company=Breng/data_0.parquet
services_part/company=DB/data_0.parquet
```

Partition pruning

DuckDB

```
D EXPLAIN ANALYZE
  SELECT count(*)
  FROM read_parquet('services_part/*/*.parquet', hive_partitioning = true)
  WHERE company = 'NS';
```

```
┌─ Summary ─────────┐
│ Total Time: 0.0020s │
│ Data Read:  32.7 kB │
└─────────────────────┘

┌─ Ungrouped Aggregate ─────────────────────┐
│ Aggregates: count_star()                  │
│ 1 row                              0µs    │
└─────────────────────┬─────────────────────┘
┌─ Table Scan ────────┴─────────────────────┐
│ Function: READ_PARQUET                    │
│ File Filters: company = 'NS'              │
│ Scanning Files: 1/15                      │
│ Total Files Read: 1                       │
│ Filename(s):                              │
│ services_part/company=NS/data_0.parquet   │
│ Row Groups Scanned: 10 / 10               │
│ 1,170,824 rows                     0µs    │
└───────────────────────────────────────────┘
```

The `File Filters` line shows the partition filter and `Scanning Files: 1/15` the result. The other fourteen files were never opened.

Remote files

DuckDB reads Parquet over HTTP (and S3, GCS, Azure) with the `httpfs` extension, which loads automatically. Only the footer and the needed column chunks are fetched, using range requests.

DuckDB

```
D EXPLAIN ANALYZE
  SELECT count(*)
  FROM 'https://blobs.duckdb.org/train_services.parquet'
  WHERE date = '2023-05-21';
```

```
┌─────────────────────────┐
│     HTTPFS HTTP Stats   │
│                         │
│        in: 16.1 KiB     │
│        out: 0 bytes     │
│          #HEAD: 1       │
│          #GET: 3        │
└─────────────────────────┘
```

```
┌─ Table Scan ──────────────────────────────────────┐
│ Function: PARQUET_SCAN                            │
│ Filters: date = '2023-05-21'::DATE                │
│ Total Files Read: 1                               │
│ Filename(s):                                      │
│ https://blobs.duckdb.org/train_services.parquet   │
│ Row Groups Scanned: 2 / 4                         │
│ 44,679 rows                                  0µs  │
└───────────────────────────────────────────────────┘
```

```
┌─ Summary ──────────┐
│ Total Time: 0.885s │
│ Data Read: 16.5 kB │
└────────────────────┘
```

Output captured on 2026-10-26 and trimmed. The 1.6 MB file was read using 16 KiB of transfer, three GET requests for

Remote and partitioned

The BTS on-time flight data for 2022 to 2024 is 17 million rows and 110 columns, about 1 GB of hive partitioned Parquet on `blobs.duckdb.org`.

DuckDB

```
D CREATE VIEW flights AS FROM read_parquet([
    'https://blobs.duckdb.org/flight-data-partitioned/Year=2022/data_0.parquet',
    'https://blobs.duckdb.org/flight-data-partitioned/Year=2023/data_0.parquet',
    'https://blobs.duckdb.org/flight-data-partitioned/Year=2024/data_0.parquet'
  ], hive_partitioning = true);
```

DuckDB

```
D SELECT Month, count(*) AS n, round(avg(DepDelay), 1) AS delay
  FROM flights
  WHERE Year = 2024 AND Origin = 'RDU'
  GROUP BY ALL ORDER BY Month;
```

```
┌──────────────────────────────┐
│        HTTPFS HTTP Stats     │
│           in: 11.1 MiB       │
│             #HEAD: 2         │
│             #GET: 148        │
└──────────────────────────────┘

┌─ Summary ─┐
│ Total Time:  2.58s │
│ Data Read: 11.7 MB │
└────────────┘
```

```
┌─ Table Scan ─────────────────────────────────┐
│ Function: READ_PARQUET                       │
│ Projections: Month, DepDelay                 │
│ Filters: Origin = 'RDU'                      │
│ File Filters: Year = 2024                    │
│ Scanning Files: 1/3                          │
│ Row Groups Scanned: 48 / 48                  │
│ 29,578 rows                          12.81s  │
└──────────────────────────────────────────────┘
```

Captured output, trimmed. Partition pruning, projection pushdown, and filter pushdown together turn 1 GB into 11 MiB

Which format when

Format	Use it for
CSV	interchange with everything, small data, one time loads
Parquet	large data at rest, sharing between tools and languages, cloud storage
DuckDB database	data you query repeatedly from DuckDB, updates and transactions, indexes
Partitioned Parquet	very large data filtered by a few columns (dates, regions)

Whatever the format, sort by the columns you filter on, keep files between 100 MB and a few GB, and avoid thousands of tiny files.

Query plans

How a query runs

1. The parser turns the SQL text into a syntax tree.
2. The binder resolves names against the catalog (tables, files, functions) and checks types.
3. The result is a logical plan, a tree of relational operators (scan, filter, project, join, aggregate) that describes what to compute.
4. The optimizer rewrites the logical plan into an equivalent one that is cheaper to run.
5. The physical planner picks an implementation for each operator (hash join, sort, …) and the execution engine runs it in parallel over vectors of rows.

`EXPLAIN` shows step 5, the physical plan. `explain_output` controls what is shown.

Logical and physical plans

```
D SET explain_output = 'optimized_only';
```
DuckDB

```
D EXPLAIN
  SELECT station_name, count(*) AS n
  FROM 'train_services.parquet'
  WHERE type = 'Intercity'
  GROUP BY ALL;
```
DuckDB

```
┌─ Projection ──────────────┐
│ Expressions: #0, n        │
│ ~48,161 rows              │
└─────────────┬─────────────┘
┌─ Aggregate ─┴─────────────┐
│ Groups: station_name      │
│ Expressions: count_star() │
│ ~48,161 rows              │
└─────────────┬─────────────┘
┌─ Parquet Scan ┴───────────┐
│ Filters: type = 'Intercity'
│ ~76,191 rows              │
└───────────────────────────┘
```

```
D SET explain_output = 'physical_only';
```
DuckDB

```
D EXPLAIN
  SELECT station_name, count(*) AS n
  FROM 'train_services.parquet'
  WHERE type = 'Intercity'
  GROUP BY ALL;
```
DuckDB

```
┌─ Hash Group By ───────────┐
│ Groups: #0                │
│ Aggregates: count_star()  │
│ ~48,161 rows              │
└─────────────┬─────────────┘
┌─ Projection ┴─────────────┐
│ Projections: station_name │
│ ~76,191 rows              │
└─────────────┬─────────────┘
┌─ Parquet Scan ┴───────────┐
│ Function: PARQUET_SCAN    │
│ Projections: station_name │
│ Filters: type = 'Intercity'
│ ~76,191 rows              │
└───────────────────────────┘
```

The logical plan is already optimized. The physical plan adds the implementation of each operator.

Estimates and actuals

The ~ numbers in `EXPLAIN` are estimates, `EXPLAIN ANALYZE` reports what actually happened.

DuckDB

```
D EXPLAIN SELECT station_name, count(*) AS n
  FROM 'train_services.parquet' WHERE type = 'Intercity'
  GROUP BY ALL ORDER BY n DESC LIMIT 5;
```

```
┌─ Top N ──────────────────────┐
│ Top: 5                       │
│ Order By: n DESC             │
│ ~5 rows                      │
└──────────────┬───────────────┘
┌─ Hash Group By ──────────────┐
│ Groups: #0                   │
│ Aggregates: count_star()     │
│ ~48,161 rows                 │
└──────────────┬───────────────┘
┌─ Projection ─┴───────────────┐
│ Projections: station_name    │
│ ~76,191 rows                 │
└──────────────┬───────────────┘
┌─ Parquet Scan ─┴─────────────┐
│ Function: PARQUET_SCAN       │
│ Projections: station_name    │
│ Filters: type = 'Intercity'  │
│ ~76,191 rows                 │
└──────────────────────────────┘
```

DuckDB

```
D EXPLAIN ANALYZE SELECT station_name, count(*) AS n
  FROM 'train_services.parquet' WHERE type = 'Intercity'
  GROUP BY ALL ORDER BY n DESC LIMIT 5;
```

```
┌─ Summary ─┐
│ Total Time: 0.0028s │
│ Data Read: 181.7 kB │
└─────────────┘

┌─ Top N ──────────────────────────────┐
│ Top: 5                               │
│ Order By: n DESC                     │
│ 5 rows                      10.0ms   │
└──────────────────────────────────────┘
                   │
┌─ Hash Group By ──┴───────────────────┐
│ Groups: #0                           │
│ Aggregates: count_star()             │
│ 159 rows                        0µs  │
└──────────────────┬┬──────────────────┘
┌─ Projection ─────┴┴──────────────────┐
│ Projections: station_name            │
│ 83,289 rows                     0µs  │
└──────────────────┬┬──────────────────┘
┌─ Table Scan ─────┴┴──────────────────┐
│ Function: PARQUET_SCAN               │
│ Projections: station_name            │
│ Filters: type = 'Intercity'          │
│ Total Files Read: 1                  │
│ Filename(s): train_services.parquet  │
│ Row Groups Scanned: 4 / 4            │
│ 83,289 rows                     0µs  │
└──────────────────────────────────────┘
```

The optimizer

The optimizer is a sequence of rewrite passes over the logical plan, 46 of them in DuckDB 2.0 (`FROM duckdb_optimizers()` lists them). The important ones,

- filter pushdown, move filters as close to the scan as possible (into it when the scan supports it),
- projection pushdown (`unused_columns`), drop columns that nothing above needs,
- join ordering, choose which tables to join first and which side to build the hash table on, using cardinality estimates,
- statistics propagation, carry min / max and null information up the plan to tighten the `~` estimates. Parquet files have no distinct value counts, which is why the previous slide estimated 48,161 groups against 159 actual,
- top N and limit pushdown, turn `ORDER BY ... LIMIT n` into a `Top N` operator that never sorts the full result.

Switching a pass off

DuckDB

```
D SET disabled_optimizers = 'filter_pushdown';
```

DuckDB

```
D EXPLAIN
  SELECT station_name, count(*) AS n
  FROM 'train_services.parquet'
  WHERE type = 'Intercity'
  GROUP BY ALL;
```

```
┌─ Hash Group By ──────────────────┐
│ Groups: #0                       │
│ Aggregates: count_star()         │
│ ~69,055 rows                     │
└──────────────────┬───────────────┘
┌─ Projection ─────┴───────────────┐
│ Projections: station_name        │
│ ~76,191 rows                     │
└──────────────────┬───────────────┘
┌─ Projection ─────┴───────────────┐
│ Projections: #1                  │
│ ~76,191 rows                     │
└──────────────────┬───────────────┘
┌─ Filter ─────────┴───────────────┐
│ Expression: type = 'Intercity'   │
│ ~76,191 rows                     │
└──────────────────┬───────────────┘
┌─ Parquet Scan ───┴───────────────┐
│ Function: PARQUET_SCAN           │
│ Projections: type,               │
```

```
│              station_name              │
│ ~380,959 rows                          │
└────────────────────────────────────────┘
```

`disabled_optimizers` removes a pass. Without filter pushdown the previous slide's query gains a separate `Filter` and the scan returns all 380,959 rows.

Switching off another

Without projection pushdown (`unused_columns`) the scan no longer lists `Projections`, every column of the file is read.

DuckDB

```
D SET disabled_optimizers = 'unused_columns';
```

DuckDB

```
D EXPLAIN
  SELECT station_name, count(*) AS n
  FROM 'train_services.parquet'
  WHERE type = 'Intercity'
  GROUP BY ALL;
```

```
┌─ Hash Group By ──────────────┐
│ Groups: #0                   │
│ Aggregates: count_star()     │
│ ~48,161 rows                 │
└──────────────┬───────────────┘
┌─ Projection ─┴───────────────┐
│ Projections: station_name    │
│ ~76,191 rows                 │
└──────────────┬───────────────┘
┌─ Parquet Scan ─┴─────────────┐
│ Function: PARQUET_SCAN       │
│ Filters: type = 'Intercity'  │
│ ~76,191 rows                 │
└──────────────────────────────┘
```

Joins

To look at join planning we use the TPC-H benchmark data, generated by DuckDB's `tpch` extension (`CALL dbgen(sf = 1)`). `lineitem` has 6 million rows and `orders` 1.5 million.

DuckDB
```
D ATTACH 'tpch.duckdb' AS tpch (READ_ONLY);
```

DuckDB
```
D USE tpch;
```

We will look at the plan for the following query, which joins the two largest tables with a selective filter on one of them.

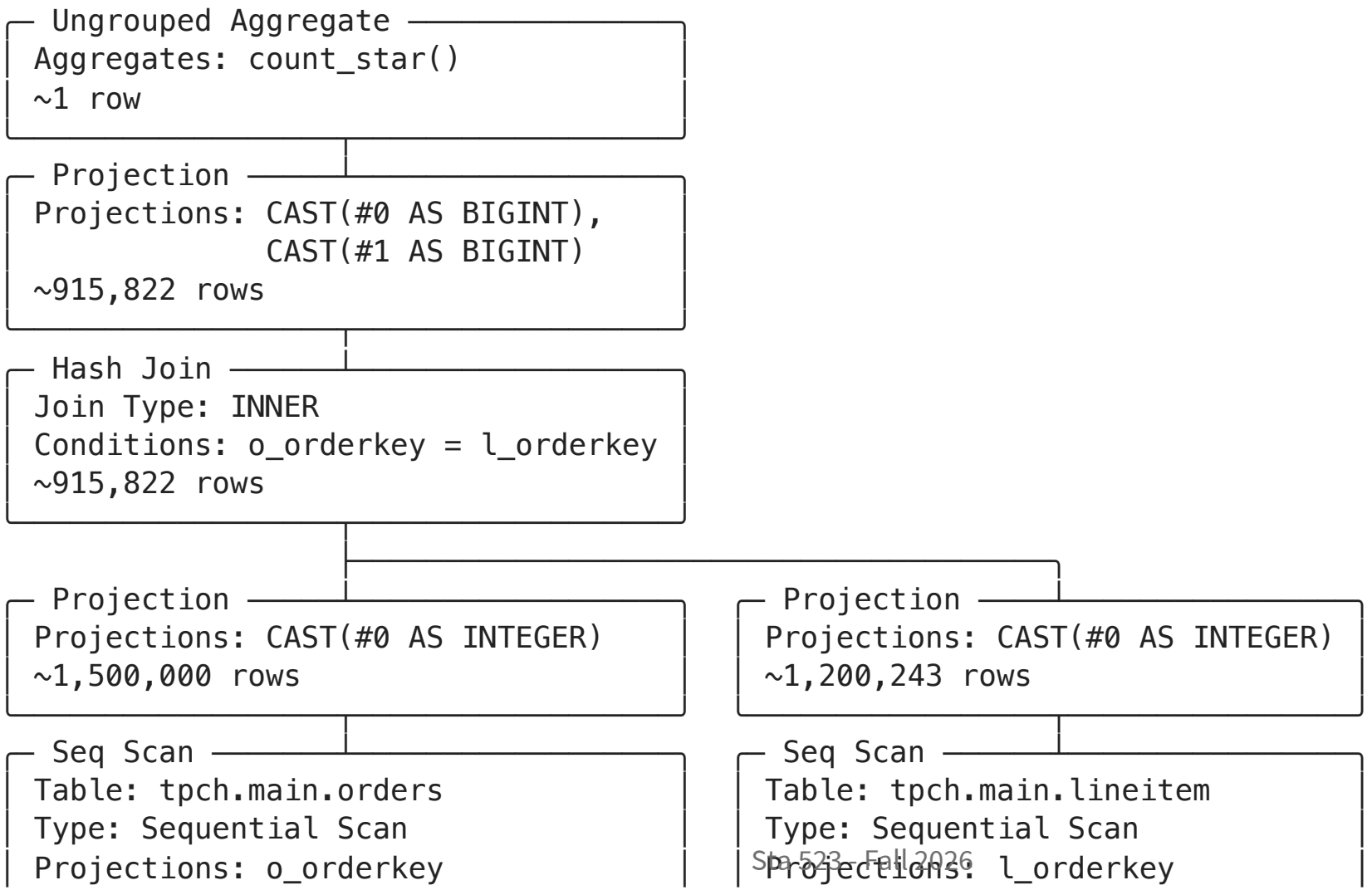
DuckDB
```
D SELECT count(*) AS n
  FROM lineitem JOIN orders ON l_orderkey = o_orderkey
  WHERE l_shipdate >= '1998-11-01';
```

```
┌──────────┐
│    n     │
│  int64   │
├──────────┤
│ 10300    │
└──────────┘
Run Time (s): real 0.010 user 0.028897 sys 0.007941
```

Join order and build side

DuckDB

```
D EXPLAIN
  SELECT count(*) AS n
  FROM lineitem JOIN orders ON l_orderkey = o_orderkey
  WHERE l_shipdate >= '1998-11-01';
```



```
~1,500,000 rows
```

```
Filters:
l_shipdate >= '1998-11-01'::DATE
~1,200,243 rows
```

A hash join builds a table from its right input and probes with the left, so the smaller input (10,300 filtered line items) belongs on the right.

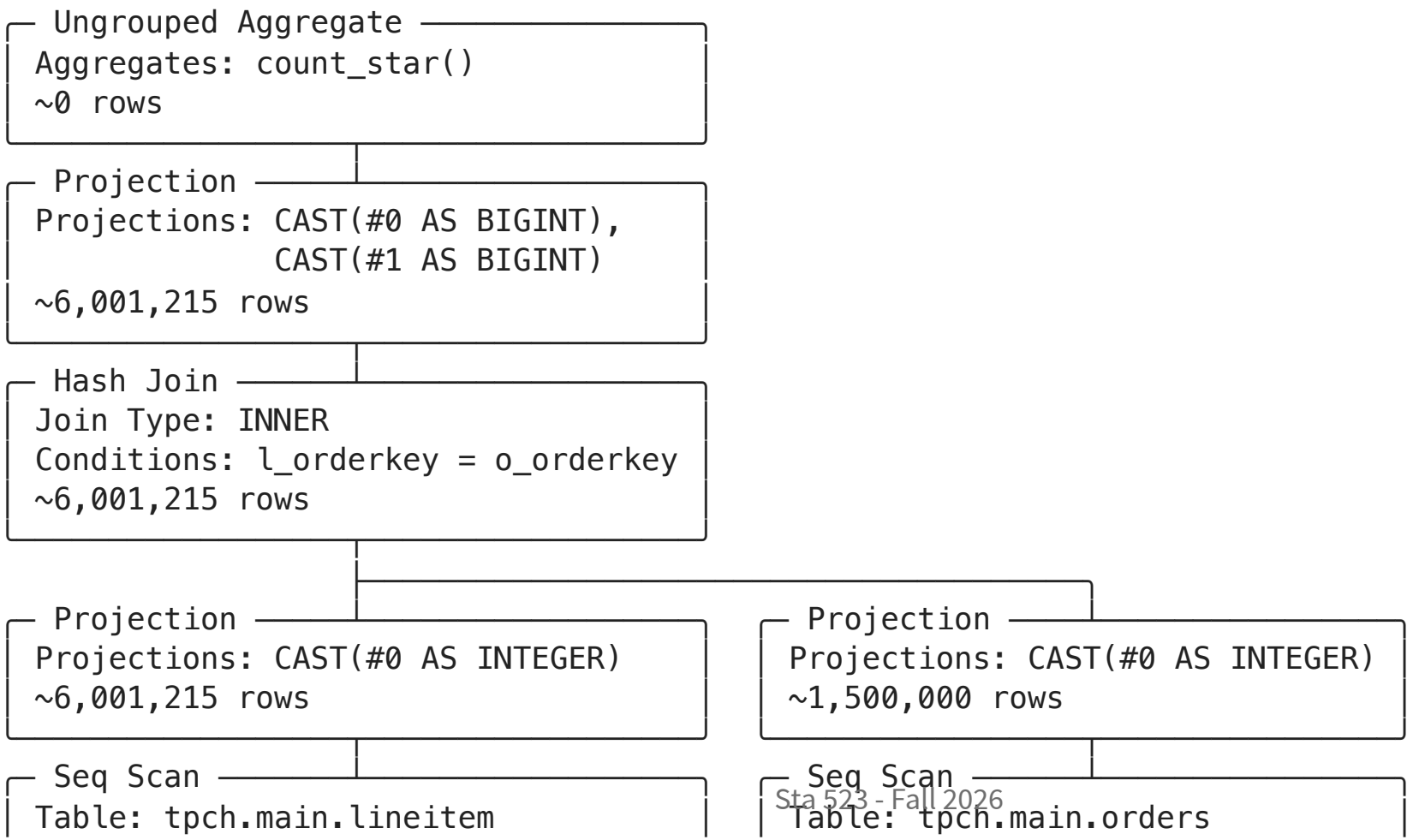
Without the join optimizer

DuckDB

```
D SET disabled_optimizers = 'join_order,build_side_probe_side';
```

DuckDB

```
D EXPLAIN
  SELECT count(*) AS n
  FROM lineitem JOIN orders ON l_orderkey = o_orderkey
  WHERE l_shipdate >= '1998-11-01';
```



```
┌────────────────────────────────────────┐
│ Type: Sequential Scan                  │
│ Projections: l_orderkey                │
│ Filters:                               │
│ l_shipdate >= '1998-11-01'::DATE       │
│ ~0 rows                                │
└────────────────────────────────────────┘
```

```
┌────────────────────────────────────────┐
│ Type: Sequential Scan                  │
│ Projections: o_orderkey                │
│ ~0 rows                                │
└────────────────────────────────────────┘
```

With `SET disabled_optimizers = 'join_order,build_side_probe_side'` the sides swap, orders are built and line items probe.

Does it matter

DuckDB

```
D SELECT count(*) AS n
  FROM lineitem JOIN orders ON l_orderkey = o_orderkey
  WHERE l_shipdate >= '1998-11-01';
```

```
┌──────────┐
│    n     │
│  int64   │
├──────────┤
│ 10300    │
└──────────┘
Run Time (s): real 0.014 user 0.075486 sys 0.023721
```

On two tables the difference is a few milliseconds. With five or six tables the number of possible join orders is in the thousands and a bad one can be slower by orders of magnitude, which is why the optimizer searches them using its cardinality estimates.

Profiling

- `.timer on` in the CLI reports wall clock time for each statement.
- `EXPLAIN ANALYZE` runs the query and reports rows and time per operator. The time is summed over threads, so an operator can report more time than the total.
- `EXPLAIN (FORMAT json)` (or `html`, `graphviz`, `mermaid`) gives the plan in a form that other tools can render. In the CLI, `.web` opens the last `EXPLAIN ANALYZE` in a browser.
- `CALL enable_profiling(format := 'json', save_location := 'profile.json')` records a profile for every query that follows, without changing the queries.

Exercise 2

Using the TPC-H tables, write a query that reports the average `o_totalprice` by `c_mktsegment` for orders placed in 1995 (`o_orderdate`).

1. `EXPLAIN` it. Which table is on the build side of each join and where did the date filter end up?
2. Run it with `EXPLAIN ANALYZE`. Which operator takes the most time, and how far off were the row estimates?

dbplyr and duckplyr

Two routes from dplyr to DuckDB

	`dbplyr`	`duckplyr`
How verbs are translated	into a SQL string sent to the database	into DuckDB's relational API, no SQL
Works with	any database with a backend	DuckDB only
Input	a table in the database (`tbl()`)	a data frame or a file (`as_duckdb_tibble()`, `read_parquet_duckdb()`)
Result	`collect()` to get a tibble	a data frame already, materialized when needed
Unknown R function	passed through to SQL, error at run time	falls back to dplyr in R
Semantics	SQL's (`NULL`, ordering, aggregates)	dplyr's, identical results to dplyr

`duckplyr` is the tidyverse's "drop in" DuckDB backend. The promise is dplyr code with no changes, DuckDB speed where possible, and dplyr behavior when not.

duckplyr frames

`read_parquet_duckdb()` and `as_duckdb_tibble()` return a `duckplyr_df`. Nothing is read until a result is needed, printing shows a preview.

```r
ts = duckplyr::read_parquet_duckdb("train_services.parquet")
class(ts)
```

```
[1] "prudent_duckplyr_df" "duckplyr_df"         "tbl_df"
[4] "tbl"                 "data.frame"
```

```r
(q = ts |>
  filter(type == "Intercity") |>
  summarize(n = n(), .by = station_name) |>
  arrange(desc(n)) |>
  head(3)
)
```

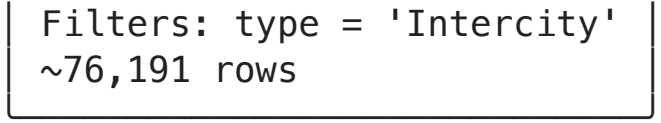
```
# A duckplyr data frame: 2 variables
  station_name           n
  <chr>              <int>
1 Utrecht Centraal    3901
2 Amsterdam Centraal  3444
3 Leiden Centraal     2170
```

The plan

`explain()` on a duckplyr frame prints the DuckDB plan, with the same pushdowns we saw from SQL.

```r
explain(q)
```

```
┌─ Top N ──────────────────────┐
│ Top: 3                       │
│ Order By: n DESC             │
│ ~3 rows                      │
└──────────────┬───────────────┘
┌─ Projection ─┴───────────────┐
│ Projections: station_name,   │
│              n               │
│ ~48,161 rows                 │
└──────────────┬───────────────┘
┌─ Hash Group By ──────────────┐
│ Groups: #0                   │
│ Aggregates: count_star()     │
│ ~48,161 rows                 │
└──────────────┬───────────────┘
┌─ Projection ─┴───────────────┐
│ Projections: station_name    │
│ ~76,191 rows                 │
└──────────────┬───────────────┘
┌─ Read Parquet ─┴─────────────┐
│ Function: READ_PARQUET       │
│ Projections: station_name    │
```


Filters: type = 'Intercity'
~76,191 rows

Prudence

A duckplyr frame is a data frame, so any R function can ask for its rows. Whether that is allowed is controlled by its prudence,

- `lavish`, materialize whenever asked (the default for `as_duckdb_tibble()`),
- `thrifty`, materialize if the result is small (the default for `read_parquet_duckdb()`),
- `stingy`, never materialize automatically, `collect()` or `as_tibble()` must be explicit.

```r
nrow(ts)
```

```
Error:
! GetQueryResult: Materialization would result in more than 125000 rows. Use `collect()` or
`as_tibble()` to materialize.
```

See the prudence article. Prudence protects against accidentally pulling a 10 GB Parquet file into R because a function

Fallback

When a pipeline uses something DuckDB cannot do, duckplyr falls back to dplyr, silently by default. `fallback_config(info = TRUE)` reports when it happens.

```r
duckplyr::fallback_config(info = TRUE)
ts_lavish = duckplyr::read_parquet_duckdb("train_services.parquet", prudence = "lavish")
```

```r
ts_lavish |> filter(date >= as.Date("2023-05-20")) |> count()
```

```
Error processing duckplyr query with DuckDB, falling back to dplyr.
Caused by error in `filter()`:
! Can't translate function `as.Date()`.

# A duckplyr data frame: 1 variable
      n
  <int>
1 92963
```

Correct but slow, the whole file went through dplyr. Hoisting the constant out fixes it.

```r
cutoff = as.Date("2023-05-20")
ts_lavish |> filter(date >= cutoff) |> count()
```

```
# A duckplyr data frame: 1 variable
      n
  <int>
1 92963
```

Fallback and prudence

A thrifty or stingy frame refuses a fallback that would materialize too much, so the mistake surfaces as an error rather than a slow silent success.

```r
ts |> filter(date >= as.Date("2023-05-20")) |> count()
```

```
Error processing duckplyr query with DuckDB, falling back to dplyr.
Caused by error:
! GetQueryResult: Materialization would result in more than 125000 rows. Use `collect()` or
`as_tibble()` to materialize.

Error in `filter()`:
! This operation cannot be carried out by DuckDB, and the input is a
  stingy duckplyr frame.
ℹ Use `compute(prudence = "lavish")` to materialize to temporary storage and
  continue with duckplyr.
ℹ See `vignette("prudence")` for other options.
Caused by error:
! GetQueryResult: Materialization would result in more than 125000 rows. Use `collect()` or
`as_tibble()` to materialize.
```

See the fallback article for what duckplyr can and cannot translate. `fallback_sitrep()` summarizes the fallbacks that

Benchmark

TPC-H query 1 on `lineitem` (6 million rows), four ways.

```r
cutoff = as.Date("1998-09-02")
q1 = function(d) {
  d |>
    filter(l_shipdate <= cutoff) |>
    summarize(
      sum_qty = sum(l_quantity), avg_disc = mean(l_discount), n = n(),
      .by = c(l_returnflag, l_linestatus)
    ) |>
    arrange(l_returnflag, l_linestatus)
}
q1_fallback = function(d) {
  d |>
    filter(l_shipdate <= as.Date("1998-09-02")) |>
    summarize(
      sum_qty = sum(l_quantity), avg_disc = mean(l_discount), n = n(),
      .by = c(l_returnflag, l_linestatus)
    ) |>
    arrange(l_returnflag, l_linestatus)
}
```

`lineitem_df` is a tibble, `lineitem_tbl` a `dbplyr` table, and `lineitem_dp` a duckplyr frame.

Benchmark results

```r
bench::mark(
  dplyr = q1(lineitem_df),
  dbplyr = q1(lineitem_tbl) |> collect(),
  duckplyr = q1(lineitem_dp) |> collect(),
  duckplyr_fallback = q1_fallback(lineitem_dp) |> collect(),
  check = FALSE, iterations = 3
) |>
  select(expression, median, mem_alloc)
```

```
# A tibble: 4 × 3
  expression          median mem_alloc
  <bch:expr>        <bch:tm> <bch:byt>
1 dplyr             735.54ms    1.03GB
2 dbplyr             24.08ms    2.94MB
3 duckplyr           38.84ms   16.91KB
4 duckplyr_fallback    9.02s    1.53GB
```

- `dbplyr` and `duckplyr` are both DuckDB doing the work, and both are much faster than dplyr on 6 million rows.
- The fallback version is slower than plain dplyr. duckplyr first materializes the whole table into R and then runs dplyr on it, paying for both.

Mixing the two

A duckplyr frame can be handed to `dbplyr` with `as_tbl()`, which exposes it as a view in duckplyr's connection. From there `show_query()`, `sql()` passthrough, and the rest of `dbplyr` are available.

```r
duckplyr::as_tbl(q) |>
  mutate(n_k = sql("round(n / 1000.0, 1)")) |>
  show_query()
```

```
<SQL>
SELECT *, round(n / 1000.0, 1) AS n_k
FROM as_tbl_duckplyr_HNSL9Ruckw
```

`compute_parquet()` runs a pipeline and writes the result to a Parquet file, returning a frame that reads from it. This is the way to checkpoint an expensive intermediate.

```r
q |> duckplyr::compute_parquet("top3.parquet") |> explain()
```

```
┌─ Read Parquet ──────────────┐
│ Function: READ_PARQUET      │
│ Projections: station_name,  │
│              n              │
│ ~3 rows                     │
└─────────────────────────────┘
```

Which one

Use `dbplyr` when

- the data already lives in a database, DuckDB or otherwise, and should stay there,
- you want to see or hand edit the SQL, or need a DuckDB function with no R equivalent,
- you need to write tables, which goes through `DBI` and `compute()`.

Use `duckplyr` when

- you have dplyr code and data frames or Parquet files, and want it faster without rewriting,
- dplyr semantics matter (results must match dplyr exactly),
- you are happy to let fallback handle the parts DuckDB cannot.

`library(duckplyr)` also calls `methods_overwrite()`, which makes every data frame in the session go through DuckDB.

Python

The ecosystem

- `duckdb` provides SQL and a relational API (method chains that build a query lazily), and reads files and remote data directly.
- `polars` is a dataframe library with its own lazy engine and optimizer, closest to DuckDB in design.
- `pandas` is the eager, in memory dataframe library most other packages accept and return.
- `pyarrow` is the in memory columnar format that all three understand. Data moves between them through Arrow without being copied into Python objects.

DuckDB can query `pandas`, `polars`, and Arrow objects in place, and return results as any of them.

Relational API

`duckdb.read_parquet()`, `duckdb.read_csv()`, and `duckdb.sql()` return a relation, a lazy description of a query. Methods add operations, nothing runs until a result is requested.

```py
r = (
  duckdb.read_parquet("train_services.parquet")
    .filter("type = 'Intercity'")
    .aggregate("station_name, count(*) AS n")
    .order("n DESC")
    .limit(3)
)
r
```

station_name varchar	n int64
Utrecht Centraal	3901
Amsterdam Centraal	3444
Leiden Centraal	2170

The plan

```py
print(r.explain())
```

```
┌─ Top N ──────────────────────────┐
│ Top: 3                           │
│ Order By: n DESC                 │
│ ~3 rows                          │
└──────────────────────────────────┘
                 │
┌─ Hash Group By ──────────────────┐
│ Groups: #0                       │
│ Aggregates: count_star()         │
│ ~48,161 rows                     │
└──────────────────────────────────┘
                 │
┌─ Projection ─────────────────────┐
│ Projections: station_name        │
│ ~76,191 rows                     │
└──────────────────────────────────┘
                 │
┌─ Parquet Scan ───────────────────┐
│ Function: PARQUET_SCAN           │
│ Projections: station_name        │
│ Filters: type = 'Intercity'      │
│ ~76,191 rows                     │
└──────────────────────────────────┘
```

`r.sql_query()` returns the generated SQL, the analogue of `show_query()`, and `r.explain(analyze=True)` of `EXPLAIN ANALYZE`.

Getting results out

A relation converts to whichever library you want to continue in.

```py
r.pl()
```

shape: (3, 2)

station_name	n
str	i64
"Utrecht Centraal"	3901
"Amsterdam Centraal"	3444
"Leiden Centraal"	2170

```py
r.df()
```

```
         station_name     n
0    Utrecht Centraal  3901
1  Amsterdam Centraal  3444
2     Leiden Centraal  2170
```

Also `to_arrow_table()`, `fetchnumpy()` (a dict of arrays), `fetchall()` (a list of tuples), and `to_arrow_reader(batch_size)` to stream a large result in batches.

Replacement scans

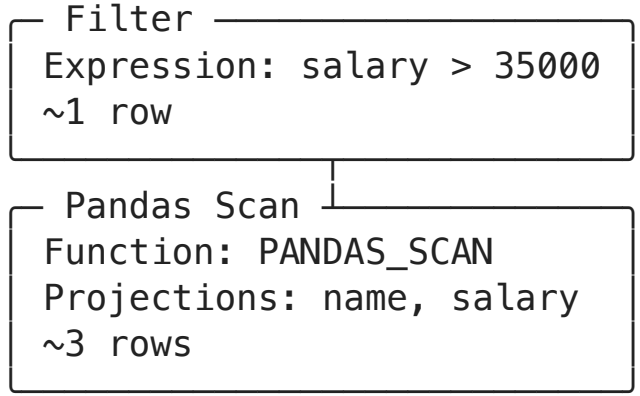
A SQL query can refer to a Python variable by name. DuckDB looks up `pandas`, `polars`, and Arrow objects in the calling scope and scans them in place.

```py
employees = pd.DataFrame({
  "name": ["Alice", "Bob", "Carol"],
  "salary": [52000, 40000, 30000]
})
```

```py
duckdb.sql(
  "FROM employees WHERE salary > 35000"
).df()
```

```
    name  salary
0  Alice   52000
1    Bob   40000
```

```py
print(duckdb.sql(
  "FROM employees WHERE salary > 35000"
).explain())
```



polars and DuckDB

A `polars` LazyFrame can be queried the same way, and the result handed back to polars.

```py
lf = pl.scan_parquet("train_services.parquet")
rel = duckdb.sql("""
  SELECT station_name, count(*) AS n
  FROM lf WHERE type = 'Intercity'
  GROUP BY ALL ORDER BY n DESC LIMIT 3
""")
rel.pl()
```

shape: (3, 2)

station_name	n
str	i64
"Utrecht Centraal"	3901
"Amsterdam Centraal"	3444
"Leiden Centraal"	2170

Pushdown into polars

The LazyFrame is scanned through Arrow, and DuckDB pushes the projection and the filter into polars, so only the needed column and rows cross the boundary.

```py
print(rel.explain())
```

```
┌─ Top N ─────────────────────┐
│ Top: 3                      │
│ Order By: n DESC            │
│ ~1 row                      │
└─────────────┬───────────────┘
┌─ Hash Group By ─────────────┐
│ Groups: #0                  │
│ Aggregates: count_star()    │
│ ~1 row                      │
└─────────────┬───────────────┘
┌─ Projection ────────────────┐
│ Projections: station_name   │
│ ~1 row                      │
└─────────────┬───────────────┘
┌─ Python Arrow Scan ─────────┐
│ Function: PYTHON_ARROW_SCAN │
│ Filters: "type" = 'Intercity'
│ Projections: station_name   │
│ ~1 row                      │
└─────────────────────────────┘
```

Same file, three readers

`timeit()` runs a function once and prints the elapsed time and the result.

```py
f, col = "yellow_tripdata_2024-01.parquet", "trip_distance"
timeit("pandas",          lambda: pd.read_parquet(f)[col].mean())
```

```
pandas              0.354 s   3.652
```

```py
timeit("pandas columns=", lambda: pd.read_parquet(f, columns=[col])[col].mean())
```

```
pandas columns=     0.036 s   3.652
```

```py
timeit("polars lazy",     lambda: pl.scan_parquet(f).select(pl.col(col).mean()).collect().item())
```

```
polars lazy         0.006 s   3.652
```

```py
timeit("duckdb",          lambda: duckdb.sql(f"SELECT avg({col}) FROM '{f}'").fetchall()[0][0])
```

```
duckdb              0.009 s   3.652
```

The engines are all fast. The difference is whether the reader was told, or worked out, that only one of the nineteen columns was needed.

Which tool

	`pandas`	`polars`	`duckdb`
Model	eager, in memory	lazy or eager, in memory	SQL, lazy, spills to disk
Larger than memory	no	streaming for some queries	yes
Parallel	no	yes	yes
Strengths	ecosystem, plotting, modeling inputs	expressions, speed, strict types	SQL, files and remote data, joins at scale
Weak spots	memory, speed, inconsistent API	smaller ecosystem	no in place mutation of frames

In practice they are used together. Read and reduce with DuckDB or polars, hand the small result to `pandas` for plotting or modeling.

Further afield

- Extensions add features to the engine, `spatial` (geometry types and GIS functions), `httpfs` and `azure` (remote storage), `json`, `iceberg` and `delta` (lakehouse table formats), and community extensions.
- `duckdb -ui` opens a notebook style interface in the browser for exploring a database.
- `ibis` is a Python dataframe API that compiles to SQL for a dozen backends, with DuckDB as the default. It plays the role `dbplyr` does in R.
- The DuckDB engine also runs in the browser (`duckdb-wasm`) and from Java, Go, Rust, Node, and Swift.

Summary

Takeaways

- Parquet stores columns separately in row groups with min / max statistics. Readers use this for projection pushdown (read only needed columns) and filter pushdown (skip row groups), and sorting a file by its filter columns makes the skipping effective.
- CSV must be parsed in full on every query. Parquet is the format for data at rest, a DuckDB database is faster still for data queried repeatedly. Partitioning lets a filter skip whole files, locally or remotely.
- `EXPLAIN` shows the physical plan with estimates, `EXPLAIN ANALYZE` adds actual rows and time per operator. The optimizer pushes filters and projections into scans and reorders joins, and `disabled_optimizers` shows what each pass contributes.
- `dbplyr` translates to SQL and works with any database. `duckplyr` uses DuckDB directly with dplyr semantics and falls back to dplyr, which is convenient and occasionally very slow.
- In Python the relational API, replacement scans, and Arrow let DuckDB, `polars`, and `pandas` share data without copies. The tool matters less than making sure only the needed columns are read.

Learning more

- DuckDB performance guide
- Querying Parquet with precision using DuckDB
- DuckDB tricks part 3 (CSV vs Parquet vs native, sorting, partitioning)
- duckplyr articles on prudence and fallback
- DuckDB Python API and relational API